(CE7456 Lecture Notes 4) Independent Query Sampling

Yufei Tao

Conventionally, a data-structure problem aims to preprocess an input set of elements into a structure such that, given a query predicate, all elements satisfying the predicate can be reported efficiently. This framework works well in practice when the query output is small. In the era of big data, however, the explosion in data volume has been accompanied by a dramatic increase in query output size. For example, while a query with selectivity 1% returns only 10000 elements on a dataset of size a million, the number surges to 10^{10} when the data cardinality reaches a trillion. In the latter scenario, even just reporting the query answer would take intolerably long.

The phenomenon has motivated *independent query sampling* (IQS) [2], which aims to return only a sample set of the query result, with the requirement that the sample sets of all queries must be mutually independent. Next, we formalize the notion over range reporting.

Let S be a set of n integers where each integer $e \in S$ is associated with an integer *weight* $w(e) > 0$. A *weighted sample* of S is a random variable X satisfying

$$\mathbf{Pr}[X = e] = \frac{w(e)}{\sum_{e' \in S} w(e')}$$

for each $e \in S$.

Given an interval $q = [x, y]$, a (traditional) *range query* reports $S_q = q \cap S$. The IQS version, however, takes two parameters:

- $q = [x, y]$ and
- an integer $s \geq 1$ (referred to as the *sample size*).

The IQS query outputs a set Q of s independent weighted samples of S_q. Furthermore, the set Q must be independent of the outputs of all previous IQS queries.

Note that even if two IQS queries have the same q, their sample results must still be independent. In fact, one can repeatedly issue the same query to obtain increasingly more samples of S_q, all of which must be mutually independent. A naive solution to IQS is to first retrieve the full query result S_q and then sample from it. However, this defeats the main purpose of IQS, i.e., reducing computation time. Instead, our objective should be to settle a query in time *significantly less* than $|S_q|$ in the typical situation where $s \ll |S_q|$. Achieving the goal demands organizing the input S in ways drastically different from the existing data structures designed to report S_q in its entirety.

1 The First Structure: Tree Sampling

Let $\mathcal{T}$ be a binary search tree (BST) on S with the following properties:

- $\mathcal{T}$ has height $O(\log n)$.
- $\mathcal{T}$ has n leaves each storing a distinct value in S as the *key*.

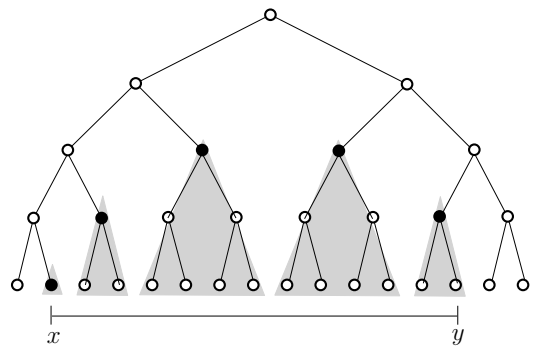

Figure 1: The black nodes are the canonical nodes of the query interval $q = [x, y]$. The shaded triangles indicate the subtrees of the canonical nodes.

- Every internal node u in $\mathcal{T}$ has two children. The *key* of u equals the smallest leaf key in the right subtree. The leaf keys in the left subtree of u are less than the key of u.

For any q, one can deploy $\mathcal{T}$ to report the entire S_q in $O(\log n + |S_q|)$ time.

Next, we explain how to adapt $\mathcal{T}$ for IQS by resorting to tree sampling. For each node u in $\mathcal{T}$, define $w(u)$ as the total weight of the leaf keys in the subtree of u. As a well-known fact, given any q, we can identify a set $\mathcal{C}$ of $O(\log n)$ *canonical nodes* in $\mathcal{T}$ such that

- the nodes in $\mathcal{C}$ have disjoint subtrees;
- the leaf keys in the subtrees of the nodes in $\mathcal{C}$ constitute S_q.

See Figure 1 for an illustration.

To draw a weighted sample of S_q, we first sample a node X from $\mathcal{C}$ such that $\mathbf{Pr}[X = u] = w(u)/\sum_{u' \in \mathcal{C}} w(u')$ for each $u \in \mathcal{C}$. Then, we perform tree sampling to obtain a leaf z from the subtree of X, and the key of z serves as a weighted sample from S_q. The above algorithm takes $O(\log n)$ time to draw a sample. Hence, s samples can be obtained in $O(s \log n)$ time. It is clear that all queries have independent outputs.

2 A Structure of $O(n \log n)$ Space

This section will reduce the time of an IQS query to $O(\log n + s)$ using a data structure of space $O(n \log n)$.

Build a BST $\mathcal{T}$ on S with the properties listed in Section 1. For each node u in $\mathcal{T}$, we use $S(u) \subseteq S$ to denote the set of elements from S that are stored at the leaves in the subtree of u. At each u, we store $w(u) = \sum_{e \in S(u)} w(e)$ and create an alias structure A_u on $S(u)$. Because A_u occupies $O(|S(u)|)$ space, the alias structures of all the nodes at the same level of $\mathcal{T}$ together use $O(n)$ space. The total space of the structure is therefore $O(n \log n)$.

Next, we explain how to answer an IQS query parameterized by interval q and sample size s. As in Section 1, we start by identifying a set $\mathcal{C}$ of canonical nodes for q (see Figure 1) in $O(\log n)$ time. Let the nodes in $\mathcal{C}$ be $u_1, u_2, ..., u_t$ for some $t = O(\log n)$. Remember that the sets $S(u_1), S(u_2), ..., S(u_t)$ constitute a partition of $S_q = q \cap S$, namely, they are mutually disjoint and their union is exactly S_q.

The next step is to determine how many samples to take from each $S(u_i)$, $i \in [1, t]$. This is an instance of weighted sampling. Specifically, a weighted sample of S_q should come from $S(u_i)$ with probability $w(u_i)/\sum_{j=1}^{t} w(u_j)$ for each $i \in [1, t]$. Let s_i be the number of weighted samples to originate from $S(u_i)$, i.e., $s_1, ..., s_t$ are random variables with sum s. We can generate these random variables in $O(t + s) = O(\log n + s)$ time as follows. First, construct an alias structure on $\mathcal{C}$ in $O(t)$ time, by treating $w(u)$ as the weight for each $u \in \mathcal{C}$. Then, we draw s weighted samples from $\mathcal{C}$ in $O(s)$ time, after which s_i $(1 \leq i \leq t)$ can be set to the number of occurrences of u_i in those samples.

For each $i \in [1, t]$, we take s_i weighted samples from $S(u_i)$ using the alias structure A_{u_i} in $O(s_i)$ time. The time of doing so for all i is $O(\sum_{i=1}^{t} s_i) = O(s)$. The overall query time is therefore $O(\log n + s)$.

From the above, we can now claim:

Lemma 1. *There is a structure of $O(n \log n)$ space that answers an IQS query in $O(\log n + s)$ time.*

3 Reducing the Space to $O(n)$

Next, we improve the space consumption to $O(n)$ using a chunking idea. Divide $\mathbb{R}$ into interior-disjoint intervals $I_1, I_2, ..., I_g$ for some $g = \Theta(n/\log n)$ satisfying the following conditions:

- If we define $\mathcal{I}_i = I_i \cap S$ for each $i \in [1, g]$, then $\mathcal{I}_1, \mathcal{I}_2, ..., \mathcal{I}_g$ are mutually disjoint and their union is S.
- $|\mathcal{I}_i| = \Theta(\log n)$ for each $i \in [1, g]$.

It is easy to obtain such intervals with sorting. We will refer to each $\mathcal{I}_i$ as a *chunk* and define $w(\mathcal{I}_i) = \sum_{e \in \mathcal{I}_i} w(e)$.

We create a structure $\mathcal{T}_{chunk}$ to support weighted range sampling at the chunk level. Specifically, the input dataset is $\mathcal{I} = \{\mathcal{I}_1, \mathcal{I}_2, ..., \mathcal{I}_g\}$ where each $\mathcal{I}_i$ $(1 \leq i \leq g)$ carries weight $w(\mathcal{I}_i)$. Given $q_{chunk} = [a, b]$ — where a and b are integers in $[1, g]$ — and an integer $s \geq 1$, a query on $\mathcal{T}_{chunk}$ returns s independent weighted samples of $\mathcal{I}_{q_{chunk}} = \{\mathcal{I}_i \mid i \in q_{chunk}\}$. It should be noted that each sample here is a chunk (rather than an element in S). We can implement T_{chunk} as a structure of Lemma 1. The space of T_{chunk} is $O(|\mathcal{I}| \log |\mathcal{I}|) = O(\frac{n}{\log n} \log n) = O(n)$.

We also build a *range sum* structure which allows us to calculate $\sum_{i=a}^{b} w(\mathcal{I}_i)$ in $O(\log n)$ time for any a and b satisfying $1 \leq a \leq b \leq g$; such a structure can be a slightly augmented BST (see Chapter 14 of [1]), which uses $O(g) = o(n)$ space. In addition, for each $i \in [1, g]$, we build an alias structure to support weighted range sampling from chunk $\mathcal{I}_i$. The alias structures of all the chunks use $O(\sum_{i=1}^{g} |\mathcal{I}_i|) = O(n)$ space in total. The overall space consumption is therefore $O(n)$.

Consider now a query with interval $q = [x, y]$ and sample size s. Let us first assume that the query is *chunk aligned*, namely, $q = I_a \cup I_{a+1} \cup ... \cup I_b$, for some a and b such that $1 \leq a \leq b \leq g$. The values of a and b can be determined in $O(\log n)$ time with binary search. Conceptually, a weighted sample from S_q can be extracted in a two-step manner. We first take a weighted sample $\mathcal{I}$ from the set of chunks $\{\mathcal{I}_a, \mathcal{I}_{a+1}, ..., \mathcal{I}_b\}$ and then acquire a weighted sample e from $\mathcal{I}$. The element

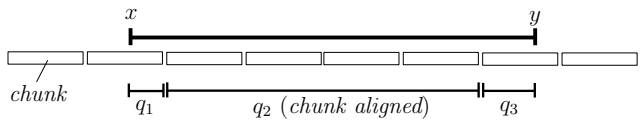

Figure 2: Query processing with chunks

e, which is sampled with probability

$$\frac{w(\mathcal{I})}{\sum_{j=1}^{g} w(\mathcal{I}_j)} \cdot \frac{w(e)}{w(\mathcal{I})} = \frac{w(e)}{\sum_{e' \in S_q} w(e')},$$

is thus a weighted sample of S_q. Motivated by this, we answer the query with the following two-level sampling strategy. First, extract s weighted samples from $\{\mathcal{I}_a, \mathcal{I}_{a+1}, ..., \mathcal{I}_b\}$, which can be done in $O(\log n + s)$ time using $\mathcal{T}_{chunk}$. For each $i \in [a, b]$, let s_i be the number of occurrences of $\mathcal{I}_i$ in those s samples. Then, we extract s_i weighted samples from $\mathcal{I}_i$ using the alias structure on $\mathcal{I}_i$ in $O(s_i)$ time. The total query cost is therefore $O(\log n + \sum_{i=a}^{b} s_i) = O(\log n + s)$.

It remains to discuss the scenario where $q = [x, y]$ can be an arbitrary interval in $\mathbb{R}$. Suppose that x and y fall in I_a and I_b, respectively, where $1 \leq a \leq b \leq g$. We can break q into three intervals q_1, q_2 and q_3 as follows:

- q_1 is the portion of q in the interior of I_a,
- $q_2 = I_{a+1} \cup I_{a+2} \cup ... \cup I_{b-1}$, and
- q_3 is the remaining portion of q after trimming q_1 and q_2 (it must hold that $q_3 \subseteq I_b$).

See Figure 2 for an illustration. Define $S_1 = S \cap q_1$, $S_2 = S \cap q_2$, and $S_3 = S \cap q_3$. For each $j \in [1, 3]$, define $w(S_j) = \sum_{e \in S_j} w(e)$. We can obtain S_1 and S_3 in their entirety in $O(\log n)$ time by reading the chunks $\mathcal{I}_a$ and $\mathcal{I}_b$ directly. For S_2, we cannot afford to enumerate its elements but can obtain $w(S_2)$ in $O(\log n)$ time from the range sum structure. To extract s weighted samples from S_q, we first determine the number s_j of samples that will come from S_j for each $j = 1, 2$, and 3 (this can be easily done in $O(s)$ time). By resorting to the alias method, the s_1 and s_3 samples from S_1 and S_3 can be fetched in $O(\log n + s_1 + s_3)$ time. Acquiring s_2 samples from S_2 requires only a chunk-aligned query, which we already know how to solve in $O(\log n + s_2)$ time. We thus conclude that the overall query time is $O(\log n + s_1 + s_2 + s_3) = O(\log n + s)$.

We have arrived at:

Theorem 2. *There is a structure of $O(n)$ space that answers an IQS query in $O(\log n + s)$ time.*

References

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition.* The MIT Press, 2009.

[2] Xiaocheng Hu, Miao Qiao, and Yufei Tao. Independent range sampling. In *PODS*, pages 246–255, 2014.